

Python Training Exams

42-style Python practice exercises organized by difficulty level.

Level 1

1) Palindrome check

Assignment name: verify_mirror

Expected files: verify_mirror.py

Allowed functions: len, .lower, .upper

Function: def verify_mirror(text: str) -> bool

Input: A string text.

Output: Boolean indicating if text is a palindrome when case-insensitive, without removing spaces or special characters.

Examples:

verify_mirror("level") -> True

verify_mirror("Level") -> True

verify_mirror("hello") -> False

2) Reverse text

Assignment name: reverse_text

Expected files: reverse_text.py

Allowed functions: len

Function: def reverse_text(text: str) -> str

Input: A string text.

Output: The reversed string.

Examples:

reverse_text("abc") -> "cba"

reverse_text("rats") -> "star"

Level 2

3) Merge and sort

Assignment name: merge_and_sort

Expected files: merge_and_sort.py

Allowed functions: none restricted

Function: def merge_and_sort(list_a: list, list_b: list) -> list

Input: Two lists of numbers (or comparable items).

Output: A new list containing all elements from both lists, sorted ascending.

Examples:

merge_and_sort([3, 1], [2, 4]) -> [1, 2, 3, 4]

merge_and_sort([5], [3, 3, 1]) -> [1, 3, 3, 5]

4) Caesar cipher with wrapping

Assignment name: shift_letters

Expected files: shift_letters.py

Allowed functions: none restricted

Function: def shift_letters(text: str, shift: int) -> str

Input: text (string) and shift (int, positive or negative).

Output: Text with letters shifted by the given amount; non-letters unchanged; wrap within A-Z and a-z separately.

Examples:

shift_letters("abc", 2) -> "cde"

shift_letters("XYZ", 2) -> "ZAB"

shift_letters("ab-cd", -1) -> "za-bc"

Level 3

5) Simple intersection

Assignment name: inter

Expected files: inter.py

Allowed functions: none restricted

Function: def inter(s1: str, s2: str) -> str

Input: Two strings s1 and s2.

Output: The characters from s1 that also appear in s2, without duplicates.

Examples:

inter("padinton", "paqueta") -> "pa"

inter("hello", "world") -> "lo"

6) Hidden Pattern (Hidenp)

Assignment name: hiddenp
Expected files: hiddenp.py
Allowed functions: none restricted
Function: def hiddenp(small: str, big: str) -> bool
Input: Two strings, small and big.
Output: True if small appears as a subsequence in big; False otherwise.
Examples:
hiddenp("abc", "axbycz") -> True
hiddenp("abc", "acb") -> False

Level 4

7) Base conversion

Assignment name: convert_base
Expected files: convert_base.py
Allowed functions: none restricted
Function: def convert_base(decimal: int, base: int) -> str
Input: decimal (non-negative int), base (2..16).
Output: Representation of decimal in the given base as a string.
Examples:
convert_base(255, 16) -> "FF"
convert_base(10, 2) -> "1010"

8) Right rotation

Assignment name: rotate_right
Expected files: rotate_right.py
Allowed functions: none restricted
Function: def rotate_right(lst: list, n: int) -> list
Input: lst (list of elements), n (non-negative).
Output: New list after a right rotation by n positions.
Examples:
rotate_right([1,2,3,4,5], 2) -> [4,5,1,2,3]
rotate_right(["a","b","c"], 1) -> ["c","a","b"]

Level 5

9) Smart queue organizer

Assignment name: smart_sort

Expected files: smart_sort.py

Allowed functions: none restricted

Function: def smart_sort(words: list) -> list

Input: List of strings.

Output: List sorted by length, lowercase alphabetical order, and vowel count.

Examples:

smart_sort(["sam", "abc", "de", "a"]) -> ["a", "de", "abc", "sam"]

10) Left rotation variant

Assignment name: rotate_left

Expected files: rotate_left.py

Allowed functions: none restricted

Function: def rotate_left(lst: list, n: int) -> list

Input: lst (list of elements), n (non-negative).

Output: List rotated left by n positions.

Examples:

rotate_left([1,2,3,4,5], 2) -> [3,4,5,1,2]

rotate_left(["a","b","c"], 1) -> ["b","c","a"]

Level 6

11) Run-length encoding

Assignment name: run_length_encode

Expected files: run_length_encode.py

Allowed functions: none restricted

Function: def run_length_encode(s: str) -> str

Input: A string s.

Output: Run-length encoded string.

Examples:

run_length_encode("aaabbcaaa") -> "a3b2ca3"

run_length_encode("abcd") -> "a1b1c1d1"

12) Multiset intersection

Assignment name: multiset_intersection

Expected files: multiset_intersection.py

Allowed functions: none restricted

Function: def multiset_intersection(s1: str, s2: str) -> str

Input: Two strings.

Output: The intersection considering multiplicity.

Examples:

multiset_intersection("aabbcc", "abbbc") -> "aabbcc"

multiset_intersection("cat", "cta") -> "cat"